

Authentication managée

Séance 12 - Clerk & Stack Auth : ne plus coder l'auth à la main

Cours 420-941-MA - Collège de Maisonneuve - Groupe 25604

build vs buy - OAuth social - Clerk - Stack Auth - middleware - synchro avec Neon

Où on en est, où on va

Du JWT fait main à l'auth managée

Acquis (S6)

- On a codé l'auth NOUS-MEMES :
- bcrypt (hachage), jwt.sign/verify
- middleware, roles, 401 vs 403
- -> très formateur
- MAIS beaucoup à gérer et sécuriser

Objectifs de la séance

- D'ABORD : les types de dossiers de routes
- (normal, [id] dynamique, (groupe))
- Puis : l'auth MANAGÉE (build vs buy)
- Clerk : mise en place complète
- Stack Auth : l'alternative open-source
- Synchroniser les users avec Neon/Prisma

L'idée de la séance. Coder l'authentification à la main (séance 6), c'est essentiel pour COMPRENDRE. Mais en vrai projet, on délègue souvent ce travail à un service spécialisé : Clerk, Stack Auth, Auth0... Ils gèrent connexion, inscription, OAuth Google/GitHub, MFA, réinitialisation de mot de passe - en quelques minutes, et de façon plus sûre. On apprend à les utiliser.

PARTIE 1

Le routing par fichiers (App Router)

Rappel : l'arborescence EST le site

Le dossier `app/` définit les URLs

En Next (App Router), on ne configure pas les routes à la main : c'est la STRUCTURE des dossiers dans `app/` qui crée les URLs. Dans chaque dossier, des fichiers SPECIAUX ont un rôle précis.

```
app/
  layout.tsx      gabarit commun (enveloppe)
  page.tsx        la page -> /
  loading.tsx     écran de chargement
  error.tsx       écran d'erreur
  not-found.tsx   page 404
  taches/
    page.tsx      -> /taches
  api/taches/
    route.ts      -> /api/taches (API)
```

dossiers

Les fichiers spéciaux

- `page.tsx` = une page visible (crée l'URL)
- `route.ts` = un point d'API (au lieu de page)
- `layout.tsx` = gabarit partagé, PERSISTE
 - entre les navigations
- `loading.tsx` = pendant le chargement
- `error.tsx` = si erreur
- `not-found.tsx` = 404
-
- Un dossier SANS `page.tsx` ni `route.ts`
- ne crée PAS d'URL (juste de l'organisation)

1) Les routes normales (statiques)

Un dossier = un segment d'URL

```
app/
page.tsx      -> /
taches/
  page.tsx    -> /taches
  archivees/
    page.tsx  -> /taches/archivees
parametres/
  page.tsx    -> /parametres
a-propos/
  page.tsx    -> /a-propos
```

dossiers

À retenir

- Le NOM du dossier = le texte de l'URL
- On IMBRIQUE en créant des sous-dossiers
- /taches/archivees =
 - dossier taches > dossier archivees
 -
- Un layout.tsx a un niveau
- enveloppe TOUT ce qui est en dessous
- (ex. une barre de navigation commune)

C'est le cas le plus simple : des pages fixes. Mais souvent l'URL contient une valeur variable (un id, un slug)... c'est la route dynamique.

2) Les routes dynamiques [id]

Une URL qui contient une valeur variable

Un dossier entre CROCHETS capture une partie variable de l'URL. Souvenez-vous de la seance 11 : la page d'edition d'une TACHE.

```
// app/tache/[id]/page.tsx (vu en seance 11)
import { prisma } from '@lib/prisma'

export default async function Page(
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params // Next 15 : async
  const tache = await prisma.tache.findUnique({
    where: { id: Number(id) } })
  return <h1>{tache?.titre}</h1>
}
```

Next

Comment ça marche

- [id] = dossier a nom variable
- /tache/42 -> id = '42'
- /tache/7 -> id = '7'
-
- Le nom entre crochets ([id])
- = la CLE dans params
- ([slug] -> params.slug)
-
- Next 15 : params est async
- -> const { id } = await params

Exactement ce qu'on a fait en S11 : un seul fichier [id]/page.tsx sert TOUTES les taches (/tache/1, /tache/2, ...). On lit l'id, on va chercher la bonne tache avec Prisma.

2 bis) Catch-all [...slug] et [[...slug]]

Attraper plusieurs segments d'un coup

```
app/blog/[...slug]/page.tsx
/blog/a      slug = ['a']
/blog/a/b/c  slug = ['a','b','c']
```

```
app/blog/[[...slug]]/page.tsx  (optionnel)
/blog                          slug = undefined (matche AUSSI)
/blog/a/b                      slug = ['a','b']
```

dossiers

La différence

- [...slug] : 1 segment ou PLUS
- params.slug est un TABLEAU
-
- [[...slug]] : les DOUBLES crochets
- = aussi le cas 'aucun segment'
- -> matche le dossier lui-meme

Lien avec la suite (auth) : Clerk utilise justement `app/sign-in/[[...sign-in]]/page.tsx` : un seul dossier gere toutes ses sous-etapes (mot de passe oublié, verification...). Vous saurez maintenant lire cette syntaxe.

3) Les groupes de routes (groupe)

Organiser SANS changer l'URL

Un dossier entre PARENTHESES n'apparaît PAS dans l'URL. Il sert à regrouper des pages et à leur donner des layouts différents.

```
app/  
  (marketing)/  
    layout.tsx      (design 'site vitrine')  
    page.tsx        -> /      (PAS /marketing !)  
    tarifs/page.tsx -> /tarifs  
  (app)/  
    layout.tsx      (design 'application')  
    dashboard/page.tsx -> /dashboard  
    taches/page.tsx  -> /taches
```

dossiers

À retenir

- (nom) = groupe, INVISIBLE dans l'URL
- /tarifs, pas /marketing/tarifs
-
- Chaque groupe peut avoir SON layout.tsx
- -> ex. un design public vs un design
- connecte, sans changer les URLs
-
- Bonus : un dossier `_prive` (underscore)
- n'est PAS routable (juste du code)

Usage typique pour l'auth : un groupe (auth) qui contient sign-in et sign-up avec un layout centre, et un groupe (app) pour les pages protégées.

Récap : les conventions de dossiers

Le tableau à garder sous les yeux

Écriture	Exemple	URL	Rôle
app/x/page.tsx	app/taches	/taches	route normale
app/x/[id]/page.tsx	app/tache/[id]	/tache/42	dynamique (params.id)
app/x/[...slug]	app/blog/[...slug]	/blog/a/b	catch-all (tableau)
app/x/[...slug]	app/sign-in/[...]	/sign-in/...	catch-all optionnel
app/(groupe)/x	app/(app)/dashboard	/dashboard	groupe (URL inchangée)
app/x/route.ts	app/api/taches	/api/taches	API (Route Handler)
app/_prive	app/_lib	(aucune)	prive, non routable

PARTIE 2

Pourquoi une auth managée ?

Rappel : coder l'auth soi-même

Ce que ça implique VRAIMENT

En seance 6, on a fait le minimum : hacher un mot de passe, signer un token, le verifier. Mais une vraie auth de production, c'est bien plus que ca.

Ce qu'on a fait (S6)

- Inscription / connexion
- Hachage bcrypt
- Token JWT + verification
- Roles (USER / ADMIN)

Ce qu'il MANQUE (et qu'il faudrait coder)

- Reinitialisation de mot de passe (email)
- Verification d'email
- Connexion Google / GitHub (OAuth)
- Double authentification (MFA), magic links
- Gestion des sessions, deconnexion partout
- Protection contre les attaques, mises a jour

Construire ou acheter ? (build vs buy)

Le choix stratégique

Auth managée : les +

- Mise en place en MINUTES
- Sécurité gérée par des experts
- OAuth social, MFA, magic links inclus
- Interfaces (login/signup) prêtes
- Sessions, revocation, appareils
- Moins de code = moins de bugs

Les points d'attention

- Dépendance à un tiers (vendor lock-in)
- Coût qui grimpe à grande échelle
- Données utilisateurs hébergées ailleurs
- Moins de contrôle fin
- -> souvent un très bon compromis

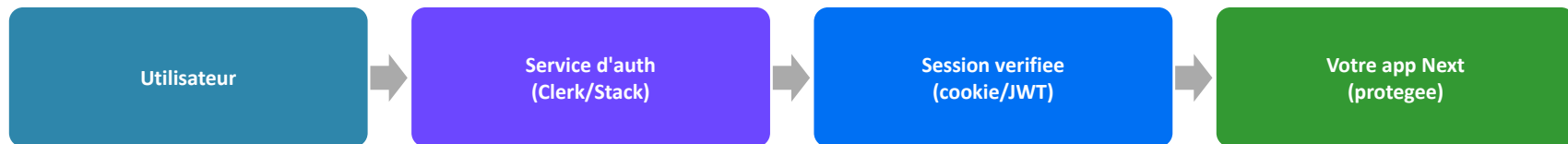
En résumé : pour la plupart des projets (et surtout au début), une auth managée fait gagner un temps énorme et évite des failles. On code l'auth soi-même quand on a des besoins très spécifiques ou des contraintes de données strictes.

PARTIE 3

Comment ça marche

Le principe d'une auth managée

Le service gère les utilisateurs, votre app fait confiance



3 pièces à connaître :

- **Le SDK cote client** : des composants prêts (boutons de connexion, profil) et des hooks.
- **Le middleware cote serveur** : il protège les routes - vérifie la session à chaque requête.
- **Les webhooks** : le service prévient votre backend (ex. 'nouvel utilisateur créé') pour synchroniser votre BD.
- **Resultat** : votre app ne stocke plus de mots de passe. Elle reçoit un utilisateur DÉJÀ vérifié.

OAuth / connexion sociale, en bref

'Se connecter avec Google'

OAuth, c'est le mecanisme derriere le bouton 'Se connecter avec Google/GitHub'. Votre app ne voit JAMAIS le mot de passe Google de l'utilisateur : Google confirme son identite et renvoie une preuve. Coder ca a la main est complexe et risque - les services managés le font pour vous, en un clic.

Ce que ça apporte

- Pas de mot de passe a gerer
- Inscription en 1 clic
- Confiance (Google/GitHub verifient)
- Moins de friction pour l'utilisateur

Avec Clerk / Stack Auth

- Activer un fournisseur = une case a cocher
- Le bouton apparait automatiquement
- Google, GitHub, Microsoft, Apple...
- -> zero code OAuth de votre cote

A retenir : l'auth managée ne fait pas que 'login/mot de passe'. Elle ouvre tout un monde (social, MFA, magic links) que vous n'auriez pas le temps de coder vous-meme.

PARTIE 4

Clerk

Clerk : l'auth clés en main

Le plus populaire dans l'écosystème Next.js

Clerk est un service d'authentification (SaaS) très intégré à React et Next.js. On l'ajoute en quelques minutes : composants prêts, middleware, gestion des utilisateurs dans un tableau de bord. Offre gratuite généreuse pour apprendre.

Prêt à l'emploi

- <SignIn/>, <UserButton/>
- Pages de connexion incluses
- Zero CSS à écrire
- Personnalisable

Complet

- Email + mot de passe
- OAuth (Google, GitHub...)
- MFA, magic links
- Organisations / rôles

Intégré Next

- clerkMiddleware
- auth() côté serveur
- ClerkProvider
- Webhooks pour la BD

Le modèle : votre app fait confiance à Clerk. Les utilisateurs vivent chez Clerk ; vous récupérez un identifiant (userId) vérifié, et vous liez vos données à cet id.

Clerk : installation & clés

3 étapes pour démarrer

1. Créer une application sur dashboard.clerk.com, puis installer :

```
npm install @clerk/nextjs
```

bash

2. Copier les clés du dashboard dans `.env.local` :

```
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_test_.... # cote client (public)  
CLERK_SECRET_KEY=sk_test_.... # cote serveur (SECRET)
```

env

À comprendre

- La cle PUBLISHABLE (NEXT_PUBLIC_) peut aller au navigateur : elle identifie votre app.
- La cle SECRET reste COTE SERVEUR (comme JWT_SECRET, comme Prisma) : jamais NEXT_PUBLIC_.
- On ne committe pas `.env.local`.

Clerk : provider & composants

Envelopper l'app, afficher connexion/profil

```
// app/layout.tsx
import {
  ClerkProvider, SignInButton, SignUpButton,
  SignedIn, SignedOut, UserButton,
} from '@clerk/nextjs'

export default function RootLayout({ children }) {
  return (
    <ClerkProvider>
      <html lang='fr'><body>
        <header>
          <SignedOut><SignInButton/><SignUpButton/></SignedOut>
          <SignedIn><UserButton/></SignedIn>
        </header>
        {children}
      </body></html>
    </ClerkProvider>
  )
}
```

Next

SignedIn / SignedOut affichent leur contenu selon l'etat. UserButton = avatar + menu (profil, deconnexion). Tout est deja stylise.

Clerk : protéger des routes (middleware)

Le gardien à l'entrée

```
// middleware.ts (à la racine)
import { clerkMiddleware, createRouteMatcher }
  from '@clerk/nextjs/server'

const estProtegee = createRouteMatcher([
  '/dashboard(.*)', '/taches(.*)',
])

export default clerkMiddleware(async (auth, req) => {
  if (estProtegee(req)) await auth.protect()
})

export const config = { matcher: [
  '/(?!_next|.*\\.\\.*)', '/(api|trpc)(.*)' ] }
```

Next

Ce que ça fait

- clerkMiddleware s'exécute
- avant chaque requête
-
- createRouteMatcher :
- quelles URLs protéger
-
- auth.protect() :
- redirige vers la connexion
- si pas authentifié
-
- = votre 'authentifiant'
- de la S6, mais gère

Clerk : lire l'utilisateur côté serveur

auth() et currentUser()

```
// dans un Server Component / Route Handler / action
import { auth, currentUser } from '@clerk/nextjs/server'
```

```
export default async function Page() {
  const { userId } = await auth()
  if (!userId) return <p>Non connecté</p>

  const user = await currentUser()
  return <p>Bonjour {user?.firstName} !</p>
}
```

Next

Les 2 fonctions

- `auth()` : rapide,
- donne le `userId`
- (l'id vérifié par Clerk)
-
- `currentUser()` : l'objet
- complet (nom, email...)
-
- Côté serveur uniquement
- -> jamais dans un client

`userId` est l'équivalent du 'sub' de votre token JWT (S6) - sauf que c'est Clerk qui l'a vérifié. Vous liez ensuite vos données (tâches, commandes...) à ce `userId`.

Clerk + Prisma/Neon : synchroniser

Lier les utilisateurs Clerk à votre base

Les utilisateurs vivent chez Clerk, mais vos DONNEES (taches, achats...) sont dans Neon. Il faut faire le pont : garder une ligne User dans votre base, identifiée par le clerkId.

Option A - à la volée (simple)

- A la 1re visite connectée,
- on crée/retrouve la ligne :
- `prisma.user.upsert({`
- `where: { clerkId: userId } }`
- `->` facile pour un cours

Option B - webhooks (robuste)

- Clerk appelle votre API quand
- un user est créé / supprimé
- `app/api/webhooks/clerk/route.ts`
- `->` synchro fiable, temps réel
- `->` l'approche de production

```
// Option A, dans une action/page serveur
const { userId } = await auth()
await prisma.user.upsert({ where: { clerkId: userId! },
  update: {}, create: { clerkId: userId! } })
```

Next

PARTIE 5

Stack Auth : l'alternative open-source

Stack Auth : l'alternative open-source

Le même service, mais que vous pouvez héberger

Stack Auth (@stackframe/stack) offre les memes fonctions que Clerk - connexion, OAuth, gestion des utilisateurs - mais il est OPEN-SOURCE. Vous pouvez l'utiliser en mode cloud (comme Clerk) OU l'heberger vous-meme (self-host).

Ses atouts

- Open-source (code visible)
- Self-hostable : vous gardez les donnees
- Composants React prêts
- Integration Next.js soignee
- Gratuit en self-host

Quand le choisir

- Besoin de controler les donnees
- Eviter le vendor lock-in
- Contraintes de confidentialite
- Envie de comprendre / bidouiller
- -> plus recent que Clerk

L'idée a retenir : meme concept d'auth managée, mais avec le controle de l'open-source. Les etapes d'integration ressemblent beaucoup a celles de Clerk.

Stack Auth : mise en place

Assistant, provider, handler, hook

1. Assistant d'installation (cree la config automatiquement) :

```
npx @stackframe/init-stack@latest
```

bash

2. Clés dans .env.local :

```
NEXT_PUBLIC_STACK_PROJECT_ID=...  
NEXT_PUBLIC_STACK_PUBLISHABLE_CLIENT_KEY=...  
STACK_SECRET_SERVER_KEY=...      # SECRET, cote serveur
```

env

Ce que l'assistant met en place

- Un StackProvider dans app/layout.tsx (comme ClerkProvider)
- Une route app/handler/[...stack]/page.tsx avec <StackHandler/> (connexion, inscription...)
- Cote client : le hook useUser() ; cote serveur : stackServerApp.getUser()
- -> memes idees que Clerk : provider + pages geres + lecture de l'utilisateur

PARTIE 6

Comparer & choisir

Auth managée vs faite main (S6)

Le tableau de décision

Critère	JWT fait main (S6)	Auth managée (Clerk/Stack)
Mise en place	longue (beaucoup de code)	quelques minutes
OAuth social (Google...)	a coder (complexe)	inclus, une case a cocher
MFA / magic link	a coder	inclus
UI login / signup	a construire	composants prêts
Securite	votre responsabilite	geree par le service
Cout	gratuit	gratuit puis payant a l'echelle
Donnees utilisateurs	votre base	chez le service (+ synchro)
Ideal pour	apprendre, cas tres specifiques	la plupart des projets reels

Clerk vs Stack Auth

Deux bonnes options, deux philosophies

Critère	Clerk	Stack Auth
Modele	SaaS (héberge par Clerk)	open-source, self-hostable
Donnees	chez Clerk	chez vous (si self-host)
Maturite	tres mature, tres populaire	plus recent, en croissance
Prix	gratuit puis payant	gratuit en self-host / cloud
Integration Next	excellente	excellente

Pour ce cours et pour demarrer vite : Clerk (le plus documente). Si vous voulez le controle des donnees ou l'open-source : Stack Auth. Les CONCEPTS sont identiques.

Sécurité & bonnes pratiques

Managé ne veut pas dire 'oublier la sécurité'

- **La cle SECRET reste cote serveur** (comme JWT_SECRET, comme Prisma) - jamais NEXT_PUBLIC_.
- **Verifier l'auth COTE SERVEUR** (middleware, auth()) - cacher un bouton ne suffit pas.
- **Ne jamais faire confiance a un id venu du client** -> toujours relire userId via auth() cote serveur.
- **Synchroniser proprement** les users vers votre base (webhooks en production).
- **Les roles / permissions** peuvent venir du service (Clerk Organizations) ou de votre base.
- **On ne stocke PLUS de mots de passe** -> une source de risque en moins.

Exercice & récap

À vous de jouer

Exercice (guide détaillé fourni)

- Créer une app Next + un compte Clerk
- Installer @clerk/nextjs + clés
- ClerkProvider + boutons + UserButton
- middleware : protéger /dashboard
- Lire l'utilisateur avec auth()
- Synchroniser avec Neon/Prisma (upsert)
- Bonus : la même chose avec Stack Auth

À retenir

- Auth managée = déléguer à un service
- Clerk : SaaS, clé en main, très populaire
- Stack Auth : open-source, self-hostable
- SDK client + middleware + webhooks
- userId vérifie -> lie à votre base
- Clé secrète = server-only, toujours

Le détail complet (créations de compte, installations, code) est dans le guide : EXERCICE-CLERK-STACKAUTH.md.



Questions ?

Prochaine étape : ajouter Clerk à votre app Next.

Vous avez codé l'auth. Maintenant, vous savez aussi la déléguer.